Information Technology University



Manual

for

CS220: Artificial Intelligence

Course Instructor : **Husnain Haider**
Semester Fall 2023 : **3**
Teaching Assistant : **Muhammad Ahmad**
Teaching Assistant : **Moeed Ahmad**
Teaching Assistant : **Muzamil Ali**
Teaching Assistant : **Abdul Rehman Ameen**
Teaching Assistant : **Muhammad Anas**

Student Name :
Student Roll No. :

Department of Computer Science ITU, Lahore, Pakistan

Lab-09

Lab Manual: Hill Climbing Algorithm

1.1 Hill Climbing Algorithm

```
Function hill ()
{
     queue = [];     // initialize an empty queue
     state = root_node;   // initialize the start state
     while (true)
     {
          if is_goal (state)
               then return SUCCESS
          else
          {
               sort (successors (state));
               add_to_front_of_queue (successors (state));
          }
          if queue == []
               then report FAILURE;
          state = queue [0]; // state = first item in queue
          remove_first_item_from (queue);
     }
}
```

1.1.1 Simple Hill Climbing

Simple hill climbing is the simplest way to implement a hill-climbing algorithm. It only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and set it as a current state. It only checks it's one successor state, and if it finds better than the current state, then move else be in the same state.

- **Step 1:** Evaluate the initial state, if it is goal state then return success and Stop.
- **Step 2:** Loop Until a solution is found or there is no new operator left to apply.
- **Step 3:** Select and apply an operator to the current state.
- **Step 4:** Check new state:
    i. If it is goal state, then return success and quit.
    ii. else if it is better than the current state then assigns new state as a current state.
    iii. else if not better than the current state, then return to step 2.
- **Step 5:** Exit.

1.1.2 Steepest-Ascent hill climbing

The Steepest-Ascent algorithm is a variation of the simple hill-climbing algorithm. This algorithm examines all the neighboring nodes of the current state and selects one neighbor node which is closest to the goal state. This algorithm consumes more time as it searches for multiple neighbors.

1.1.3 Stochastic hill climbing

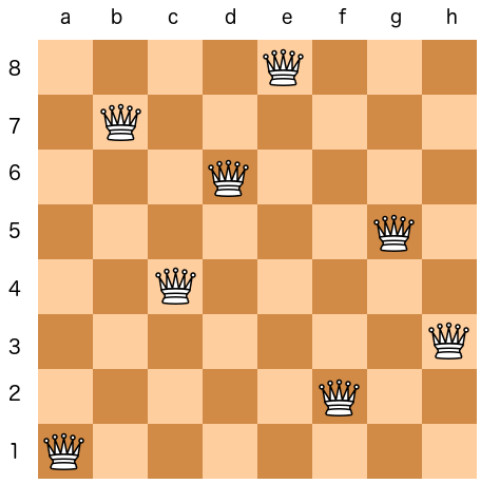
Stochastic hill climbing does not examine for all its neighbors before moving. Rather, this search algorithm selects one neighbor node at random and evaluate it as a current state or examine another state.

Task 01:
The Eight Queens Puzzle is a famous problem that originates from chess. The challenge is to place eight queens on a standard 8x8 chessboard in such a way that no two queens can attack each other.

In chess, a queen can move horizontally across rows, vertically along columns, and diagonally. So, to solve this puzzle, you need to find an arrangement where:

- No two queens are in the same row.
- No two queens are in the same column.
- No two queens can attack each other along any diagonal.

This means that every queen must be positioned in a unique row and column, and their diagonal paths must also be safe from one another.



Solve the problem using Hill Climbing Search algorithm (Steepest - Ascent Hill Climbing)

Here are the following functionalities that can help you to solve this problem
1. Initialize_board
Purpose: Generates a random starting configuration of queens, placing one queen in each row without initial column conflicts.
Output: A list of integers where index represent Row and value represent Column
[3, 6, 2, 7, 0, 4, 1, 5]

2. Calculate_conflicts
Purpose: Calculates the number of queen conflicts (same column or diagonal attacks) on the given board.

3. Generate_neighbors
Purpose: Creates neighboring configurations by moving each queen to a different column in its row.

4. Find_best_neighbor
Purpose: Evaluates all neighbors and returns the one with the fewest conflicts.

5. Hill_climb
Purpose: Implements the hill-climbing algorithm. Iteratively moves to the best neighbor until no further improvements can be made.

You can make other functions too for you own need

Submission Instructions:
Save your notebook with the naming scheme as **“Lab09_BSCS23___”**
Submit your file to your Google Classroom